Step at a Time' " dataset comprises recordings from 18 participants (ages 18–27, mean age 21.0 ± 2.3 years, 8 reported female) who listened to two consecutive stories within the same scanning run. The "Slumlord" story is 15:03 minutes long (602 TRs, 2,715 words), while "Reach for the Stars One Small Step at a Time" is 13:45 minutes long (550 TRs, 2,629 words).

Following the data quality guidelines from the original paper, we excluded noisy recordings, resulting in a total of 150 valid recordings from 147 unique individuals, as three participants contributed to more than one dataset in separate scanning sessions.

We used the preprocessed unsmoothed data without applying any additional preprocessing steps. Preprocessing in the Narratives collection was performed using the fMRIPrep pipeline, which included susceptibility distortion correction, slice-timing correction, spatial normalization to the "fsaverage" brain template, and projection onto the cortical surface. Both smoothed and unsmoothed versions of the data are available, and no temporal filtering was applied. The fMRI data were sampled at a repetition time (TR) of 1.5 seconds.

4.1.1 Brain Parcellation

All brain mapping analyses were conducted at the voxel level. However, for certain comparisons, we computed the average brain score across voxels within ROIs (Fig. 3b and 4b) to facilitate clearer interpretation. These ROIs were defined by grouping brain voxels based on their anatomical location using the Destrieux Atlas [57], which provides a standardized parcellation of the cortex into 75 regions per hemisphere. The full names of ROIs abbreviated in our analyses are listed in Table 1.

Table 1 List of abbreviated ROIs and their full anatomical names based on the Destrieux Atlas. These ROIs were used to compute average brain scores across voxels within each region.

Abbreviation	Full Name
HS	Heschl's Sulcus
HG	Heschl's Gyrus
STG	Superior Temporal Gyrus
STS	Superior Temporal Sulcus
IFG	Inferior Frontal Gyrus
Lateral S	Lateral Sulcus (Sylvian Fissure)

4.2 Large Language Model Representations

The alignment of brain activity with both lexical and contextual embeddings (activations) from various pre-trained deep language models has been extensively explored in prior research [5, 6]. Additionally, studies have highlighted the potential of attention weights and transformations from models like GPT-2 and BERT for predicting brain activity in response to identical inputs [7, 8].

In this study, we employed attribution scores to explain the outputs of language models and compared these scores to fMRI recordings from participants exposed to the same input sentences. Specifically, we focused on three language models—GPT-2 (124 million parameters), Phi-2 (2.7 billion parameters), and Llama 2 (7 billion parameters)—chosen based on their demonstrated brain alignment, language performance, and architectural diversity.

We used pre-trained models available on `Hugging Face` [58] and extracted representations for each word in the input stories based on their preceding context. This process involved constructing sequences where each sequence included a target word along with a specified number of preceding words. Tokenization was performed using the `Hugging Face` auto tokenizer, and the tokenized sequences were fed into the models to extract word-level representations.

4.2.1 Feature Attribution

Feature attribution is a method used to explain the predictions of machine learning models by assigning importance scores to each input feature. In this study, we utilized feature attribution to investigate

the relationship between language model outputs and brain activity. This approach allowed us to evaluate the influence of preceding words on the model's next-word predictions and compare it to human predictive processing.

We evaluated four different attribution methods: Gradient Norm, Gradient $\times$ Input, Integrated Gradients, and Erasure. These methods served as explanatory representations of the models, enabling an in-depth analysis of the alignment between model predictions and neural activity.

Gradient Norm: The Gradient Norm method [25, 26] measures the sensitivity of the model's output to changes in the input token embeddings x. The importance score for each token is computed as the L_1 norm of the gradient of the output with respect to the token embedding:

$$s(x_i) = \|\nabla_{x_i} f(x)\|_1 \tag{1}$$

where $f(x)$ is the model's output, ∇_{x_i} denotes the gradient with respect to token x_i, and $\|\cdot\|_1$ represents the L_1 norm.

Gradient $\times$ Input: The Gradient $\times$ Input method [27] combines gradient-based sensitivity analysis with the magnitude of the input embeddings. It computes the importance score for a token as the dot product of the gradient of the model's output and the token embedding:

$$s(x_i) = \nabla_{x_i} f(x) \cdot x_i \tag{2}$$

To improve interpretability, the L_2 norm can be applied to the scores [28]:

$$s(x_i) = \|\nabla_{x_i} f(x) \cdot x_i\|_2 \tag{3}$$

where $\nabla_{x_i} f(x)$ is the gradient of the model's output with respect to x_i, and $\cdot$ represents the dot product.

Integrated Gradients: Integrated Gradients [29] address the saturation problem by attributing importance along a path from a baseline x' (e.g., zero or [MASK] embeddings) to the actual input x. The importance score is defined as:

$$s(x_i) = (x_i - x_i') \cdot \int_{\alpha=0}^{1} \nabla_{x_i} f(x' + \alpha(x - x'))\, d\alpha \tag{4}$$

The integral is often approximated using a Riemann sum:

$$s(x_i) \approx \frac{1}{m} \sum_{k=1}^{m} \nabla_{x_i} f\left(x' + \frac{k}{m}(x - x')\right) \cdot (x_i - x_i') \tag{5}$$

where m is the number of interpolation steps, and $\nabla_{x_i} f(x)$ is the gradient of the output with respect to x_i. For the implementation of Integrated Gradients, we used the `Captum` library [59, 60].

Erasure: Erasure-based methods [30] assess the importance of a token by removing or masking it and observing the change in the model's output. The importance score is given by:

$$s(x_i) = f(x) - f(x_{-i}) \tag{6}$$

where $f(x)$ is the model's output for the original input, and $f(x_{-i})$ is the output after removing or masking token x_i.

Feature Space Design Using Word Attribution: To compute LLM attributions, we segmented the input story into overlapping sequences using a sliding window approach. The window length was chosen based on prior work showing that embeddings of upcoming words can be reliably decoded from brain activity up to a second before they are perceived [36]. To capture this predictive structure, we used a sliding window spanning approximately one second of linguistic input, containing 10 words per window. Each window included the target word along with its 9 preceding words, with a 9-word overlap between consecutive windows.

For each input sequence, we computed importance scores for every token using the aforementioned attribution methods. These methods output a vector per token. To derive a single per-token score, we computed the L_1 norm for all methods except Gradient $\times$ Input, for which we used the L_2 norm, as this approach has demonstrated superior interpretability [28]. Tokens belonging to the same word (e.g., 'not' and 'ebook' in the tokenized representation of 'notebook') were summed to calculate a unified importance score for that word.

Since the sliding windows overlap, each word in the story is processed within 10 different windows. This results in multiple importance scores being assigned to a single word. To represent these scores, we stored all computed scores for each word in a vector associated with that word. Words appearing in fewer than 10 windows (i.e., the first and last 10 words of the story) were excluded to ensure consistent processing.

This methodology produced a feature matrix of dimensions $(W - 20) \times 10$, where W denotes the total number of words in the story, and the second dimension corresponds to the feature scores derived from the overlapping windows.

4.2.2 Layer Conductance

Layer conductance is an extension of attribution methods that provides a deeper understanding of the internal functioning of neural networks [46]. While traditional attribution methods focus on identifying which input features most influence predictions, layer conductance measures the contribution of individual hidden units within the network. This method quantifies how information flows through neurons and accumulates across layers. It is built upon Integrated Gradients [29] and assigns attributions of input features, computed by Integrated Gradients, to neurons in each layer via the chain rule. The conductance of neuron y for an input feature i is defined as:

$$cond_i^y(x) = (x_i - x_i') \cdot \int_{\alpha=0}^{1} \frac{\partial f(x' + \alpha(x - x'))}{\partial y} \cdot \frac{\partial y}{\partial x_i} \, d\alpha. \tag{7}$$

Here, x is the input, x' is the baseline, and f is the model. The term $cond_i^y(x)$ quantifies how much of the input contribution to the prediction passes through neuron y, with the total conductance for a layer computed by summing the importance scores of all neurons within that layer.

In this study, we used layer conductance to construct a feature space, employing a moving window approach similar to the one used for feature attribution. To compute conductance scores, we used the `Captum` library, which provides a set of conductance values for each word in the input sequence, corresponding to the contributions of different model layers. For example, when a sentence is fed into GPT-2, `Captum` returns a vector of 12 conductance scores per word, where each value represents the contribution of a specific model layer for that word. The conductance values were computed for each input window, resulting in vectors per layer with the same dimensions as the window. Consequently, each word in the story was represented by a vector capturing its conductance scores across all the windows in which it appeared. By concatenating these vectors, we constructed the layer conductance feature space, representing the model's internal dynamics concerning the input data.

Conductance also serves as a metric for quantifying neuron importance. While Equation 7 characterizes the function of a neuron in terms of its effect on the input, Equation 8 evaluates the overall